

## Лабораторна робота № 1

### Тема. Взаємодія та специфіка робота з GitHub. Створення першого репозиторію

**Мета:** ознайомитися з системою GitHub та навчитися користуватися терміналом Git Bash.

#### Короткі теоретичні відомості

GitHub – це інструмент, який дуже корисний для розробників програмного забезпечення. Він допомагає зберігати, спільно працювати та вдосконалювати код. Сьогодні ми поговоримо про те, чому GitHub так важливий, його переваги і недоліки.

##### Переваги GitHub.

1. Збереження коду: GitHub – це як сховище для вашого коду. Ви можете зберігати свій код тут і не більше не турбуватися про втрату даних. Якщо ваш комп'ютер зламається, ви все одно зможете отримати свій код з GitHub.

2. Спільна робота: Ви можете працювати над проектами разом з іншими розробниками навіть, якщо ви далеко від них географічно. GitHub надає зручні інструменти для спільної роботи над кодом.

3. Історія змін: GitHub відстежує всі зміни, які ви робите у своєму коді. Це допомагає зрозуміти, як розвивався проект з часом і відстежувати помилки.

4. Відкритий код: багато проектів на GitHub є відкритими, що означає, що вони вільно доступні для всіх. Це сприяє розвитку програмної галузі та навчанню.

5. Інструменти для автоматизації: GitHub надає можливості автоматизувати багато рутинних завдань, таких як тестування та розгортання коду.

##### Недоліки GitHub.

Приватність даних: незважаючи на можливість створення приватних репозиторіїв, ваш код все одно може бути доступним для певних людей, якщо ви не обережні з налаштуваннями безпеки.

Великий обсяг інформації: для новачків GitHub може здатися складним через велику кількість функцій та інформації.

Чому всі використовують GitHub.

GitHub став стандартом у світі розробки програмного забезпечення через свою надійність, зручність і спрощення спільної роботи. Він допомагає розробникам у всьому світі створювати якісний код, спільно працювати над проектами і навіть знайти роботу, демонструючи свої навички через свій профіль на GitHub.

Основні характеристики Git Bash.

1. Командний рядок Git: Git Bash надає доступ до команд Git, що дозволяє розробникам використовувати всі функції Git для створення, коміту, гілок, злиття та інших операцій управління версіями.

2. Сумісність зі синтаксисом Unix: Git Bash побудований на основі Unix-подібного терміналу, що означає, що ви можете використовувати традиційні Unix-команди в середовищі Windows.

3. Підтримка скриптів: Ви можете створювати скрипти та автоматизовані завдання за допомогою Git Bash, що робить його корисним для автоматизації рутинних завдань розробки.

4. Інтеграція з іншими інструментами розробки: Git Bash може бути інтегрований з іншими інструментами розробки, такими як редактори коду, для полегшення роботи з Git в контексті вашого проекту.

### **Порядок виконання роботи**

**Крок 1.** Спочатку необхідно зареєструватися на GitHub за посиланням ([https://github.com/signup?return\\_to=https%3A%2F%2Fgithub.com%2Fsignup&source=login](https://github.com/signup?return_to=https%3A%2F%2Fgithub.com%2Fsignup&source=login)) та завантажити Git Bash за посиланням (<https://git-scm.com/downloads>).

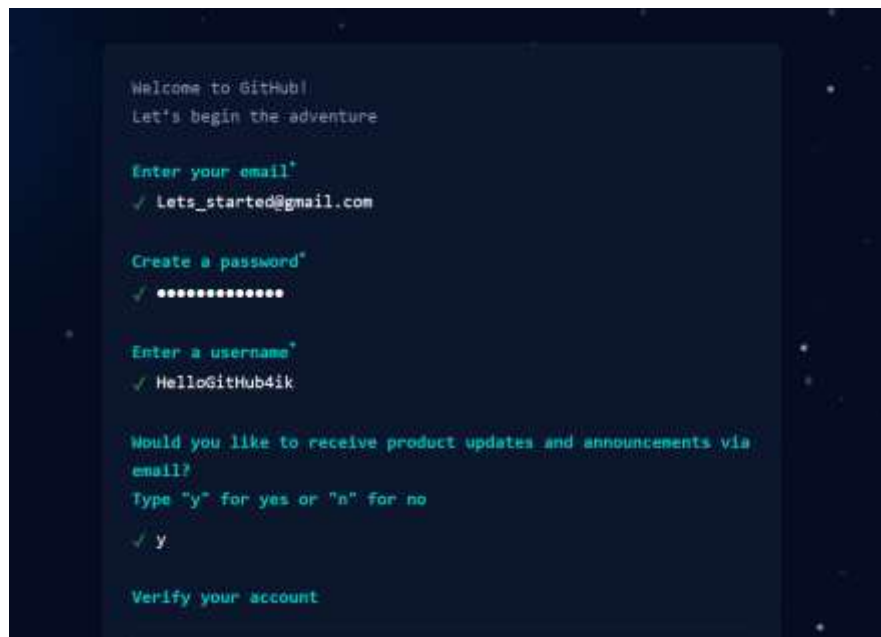


Рисунок 1.1 – Реєстрація на GitHub

Також встановіть Git на свій комп'ютер за посиланням (<https://desktop.github.com/>).

**Крок 2.** Після того як створили обліковий запис та увійшли, необхідно створити перший репозиторій.

Репозиторій, як правило, використовується для організації одного проекту. Репозиторії можуть містити теки та файли, зображення, відео, електронні таблиці та набори даних – все, що потрібно для Вашого проекту. Зазвичай репозиторії включають файл README, який містить інформацію про ваш проект. Файли README написані мовою Markdown у звичайному текстовому форматі. Ви можете скористатися цим «шпаргалкою» для ознайомлення з синтаксисом Markdown. GitHub дозволяє додати файл README в той самий час, коли ви створюєте новий репозиторій. Також GitHub пропонує інші загальні параметри, такі як ліцензійний файл, але ви не повинні вибирати їх зараз.

Ваш репозиторій «hello-world» може бути місцем, де ви зберігаєте ідеї, ресурси або навіть ділитесь та обговорюєте речі з іншими.

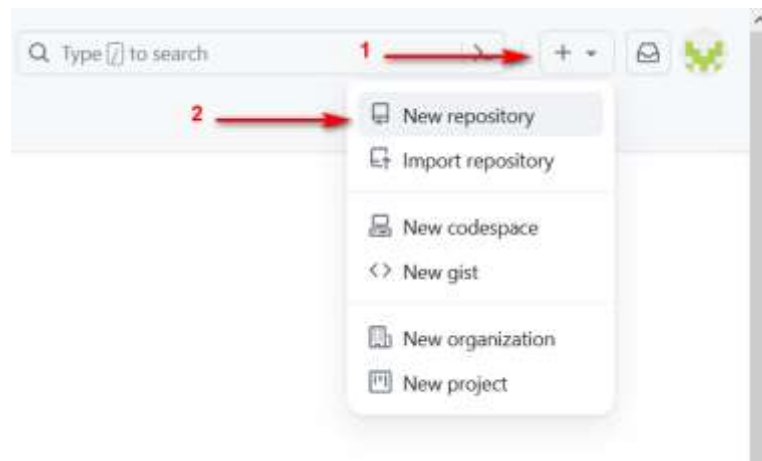


Рисунок 1.2 – Створення нового репозиторію

Щоб створити репозиторій, необхідно у верхньому правому кутку натиснути на випадаюче меню «+», а далі виберіть «**New repository**».

### Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?  
[Import a repository.](#)

Required fields are marked with an asterisk (\*).

Owner \* / Repository name \* hello-world hello-world is available.

Great repository names are short and memorable. Need inspiration? How about [glowing-giggle](#) ?

Description (optional)  
У полі "Description" введіть короткий опис.

☒ Public Anyone on the internet can see this repository. You choose who can commit.  
☐ Private You choose who can see and commit to this repository.

Initialize this repository with:  
☒ Add a README file This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore  
.gitignore template: None

Choose a license  
License: None

This will set `main` as the default branch. Change the default name in your [settings](#).

① You are creating a public repository in your personal account.

Create repository

Рисунок 1.3 – Створення опису нового репозиторію

Після того як потрапили на сторінку для створення репозиторію, необхідно:

1. У полі «Description» введіть короткий опис.
2. Виберіть, чи буде ваш репозиторій «**Public**» або «**Private**». В нашому випадку він буде «**Public**».
3. Виберіть «**Add a README file**».
4. Натисніть «**Create repository**».

### Крок 3. Створення гілки «Creating a branch».

Створення гілки дозволяє мати кілька різних версій репозиторію одночасно.

За замовчуванням у вашому репозиторії є одна гілка, названа «main», яка вважається основною. Ви можете створити додаткові гілки від гілки «main» у вашому репозиторії. Гілки можна використовувати, щоб мати кілька різних версій проекту одночасно. Це корисно, коли ви хочете додати нові функції до проекту, не змінюючи основний код. Робота, яка виконується в різних гілках, не відображатиметься в головній гілці, поки ви її не об'єднаєте, про що ми поговоримо пізніше. Гілки дозволяють експериментувати та вносити зміни перед їхнім включенням в головну гілку.

Коли ви створюєте гілку від гілки «main», ви створюєте копію або знімок «main» на той момент часу. Якщо хтось інший вніс зміни в гілку «main», поки ви працювали над своєю гілкою, ви можете включити ці оновлення.

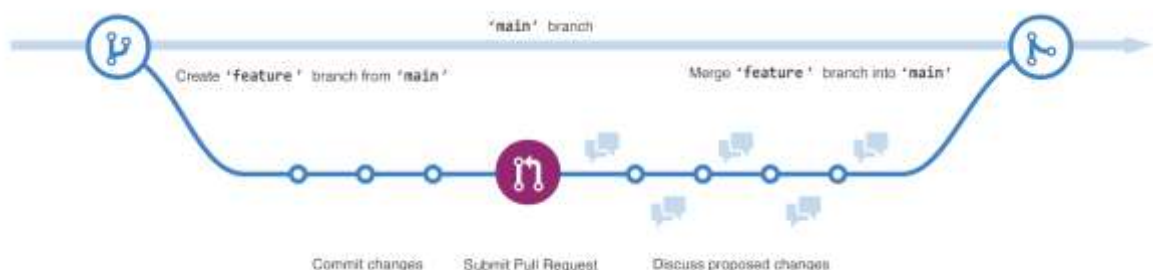


Рисунок 1.4 – Вигляд додаткової гілки

На цій діаграмі показано:

1. Гілку «main».
2. Нову гілку під назвою «feature».
3. Подорож, яку робить «feature» до її об'єднання в «main».

Різні версії файлу можуть мати вигляд. Наприклад:

- story.txt
- story-edit.txt
- story-edit-reviewed.txt

Гілки досягають схожих цілей у репозиторіях на GitHub.

У GitHub розробники, письменники та дизайнери використовують гілки для відокремлення виправлень помилок та роботи над функціями від основної (продукційної) гілки. Коли зміна готова, вони об'єднують свою гілку в головну.

Отже, коли стало зрозуміло для чого потрібні гілки, перейдемо до створення першої гілки:

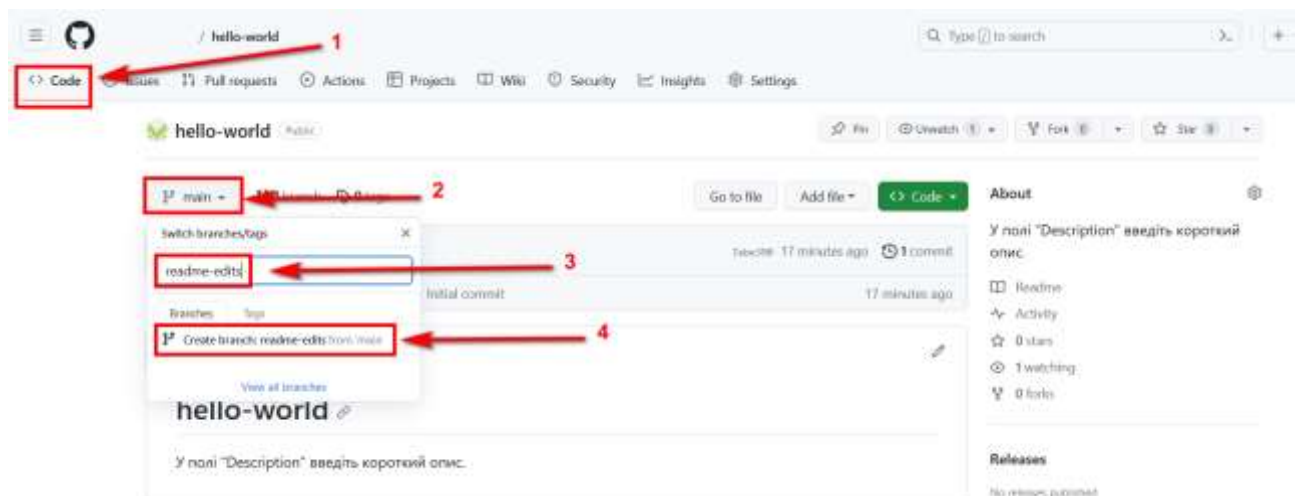


Рисунок 1.5 – Створення нової гілки

1. Натисніть на вкладці «Code» вашого репозиторію «hello-world».
2. Над списком файлів натисніть на розкривному меню, яке вказує «main».
3. Введіть назву гілки, «readme-edits», у текстове поле.
4. Натисніть «Створити гілку: readme-edits з main».

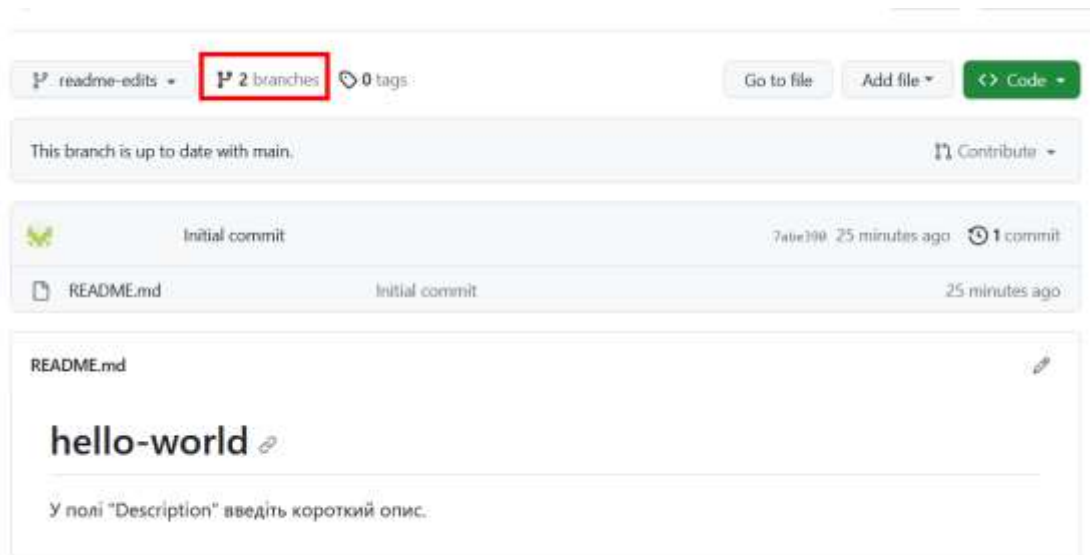


Рисунок 1.6 – Кількість гілок в репозиторії

Тепер у вас є дві гілки, «main» і «readme-edits». Зараз вони виглядають точно так само. Далі додамо зміни до нової гілки.

#### Крок 4. Створення та збереження змін.

Після створення нової гілки на попередньому кроці, GitHub перенаправив вас на сторінку «[https://github.com/Ваш\\_нікнейм/hello-world/tree/readme-edits](https://github.com/Ваш_нікнейм/hello-world/tree/readme-edits)» коду вашої нової гілки «readme-edits», яка є копією «main».

Ви можете вносити та зберігати зміни в файлах свого репозиторію. На GitHub збережені зміни називаються комітами «commit».

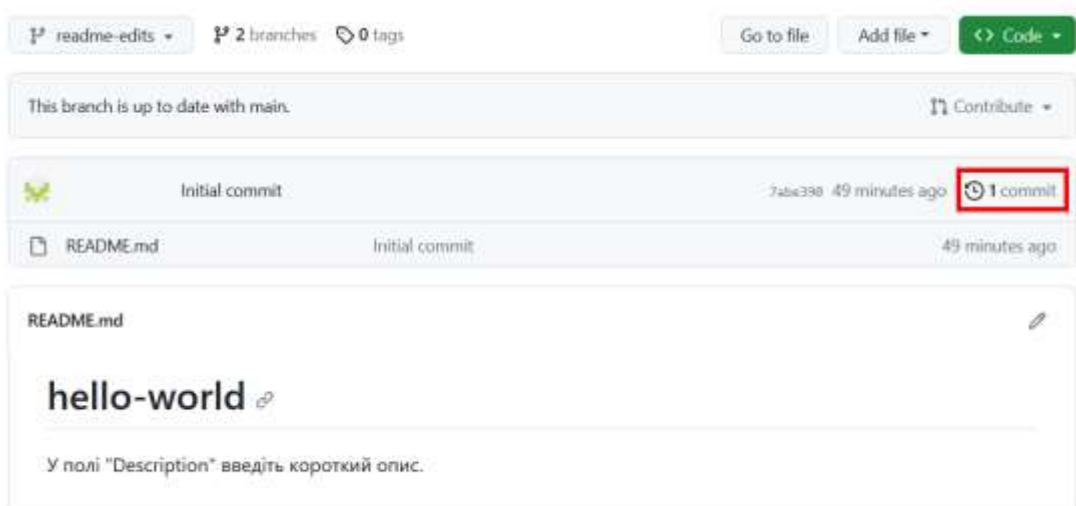


Рисунок 1.7 – Кількість комітів в репозиторії

Кожен коміт має пов'язаний з ним коментар, який є описом, що пояснює, чому була зроблена певна зміна. Коментарі до комітів зафіксують історію ваших змін, щоб інші учасники могли зрозуміти, що ви зробили і чому.

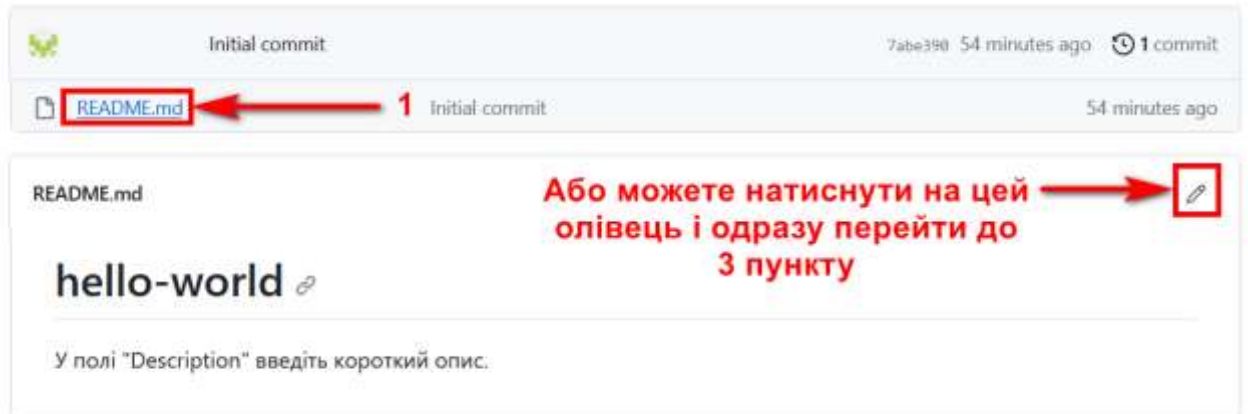


Рисунок 1.8 – Відкриття файлу для редагування

1. Під гілкою «readme-edits», яку ви створили, клацніть на файл README.md.
2. Щоб відредагувати файл, натисніть .

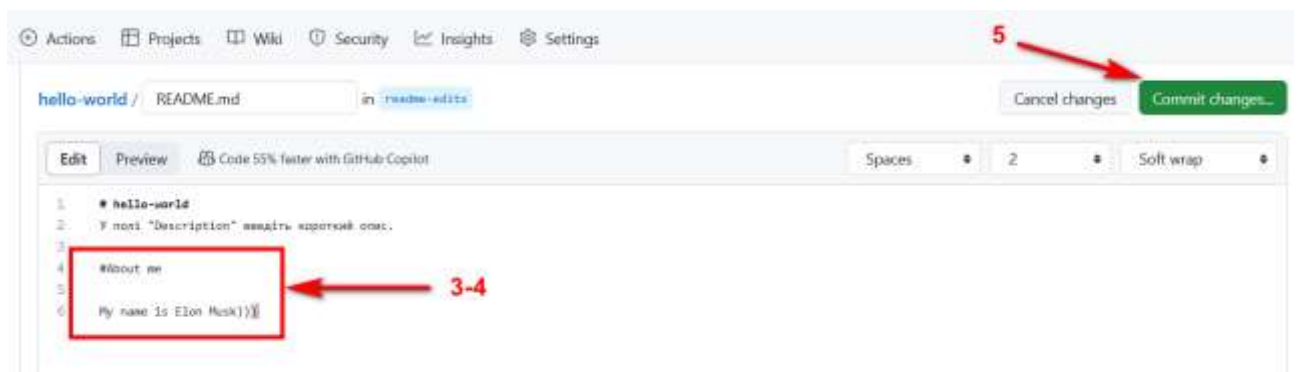


Рисунок 1.9 – Додавання нових змін в файл

3. В редакторі напишіть трохи про себе.
4. Спробуйте використовувати різні елементи Markdown.
5. Натисніть «Commit changes».

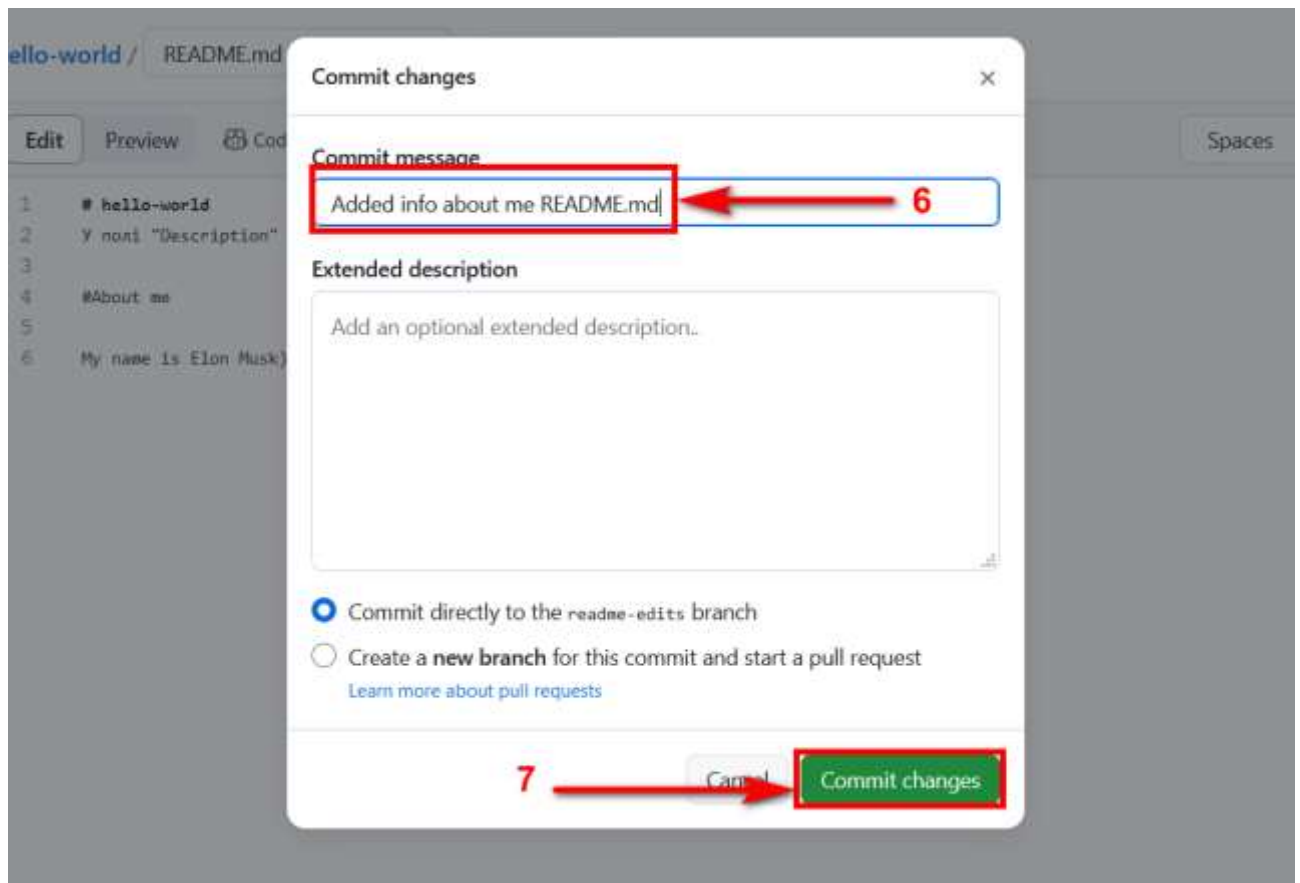


Рисунок 1.10 –Додавання нового коміту до створених змін

6. У полі «Commit changes» напишіть коментар до коміту, який описує ваші зміни.

7. Натисніть кнопку «Commit changes».

Ці зміни будуть внесені лише до файлу README у вашій гілці «readme-edits», тепер ця гілка містить контент, відмінний від головної («main»).

**Крок 5.** Відкриття запиту на включення змін «pull request».

Тепер, коли у вас є зміни в гілці, відокремленій від головної, ви можете відкрити запит на включення змін.

Запити на включення змін - це серце спільної роботи на GitHub. Коли ви відкриваєте запит на включення змін, ви пропонуєте свої зміни та запитуєте когось переглянути та включити ваш внесок та об'єднати їх у свою гілку. Запити на включення змін показують відмінності або різницю вмісту з обох гілок. Зміни, додавання та видалення показані різними кольорами.

Як тільки ви зробите коміт, ви можете відкрити запит на включення змін та розпочати обговорення, навіть до завершення написання коду.

За допомогою функції @mention GitHub у вашому повідомленні запиту на включення змін, ви можете попросити про зворотний зв'язок конкретних осіб чи команд, незалежно від того, чи знаходяться вони в сусідній кімнаті чи в інших часових поясах.

Ви навіть можете відкривати запити на включення змін у своєму власному репозиторії та об'єднувати їх самостійно. Це чудовий спосіб вивчити робочий процес GitHub перед роботою над більшими проектами.

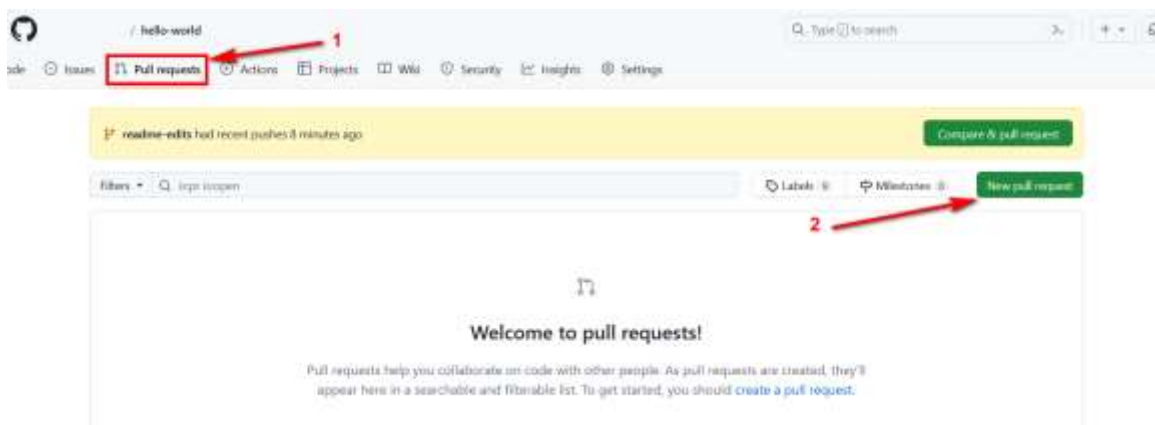


Рисунок 1.11 – Створення нового «Pull requests»

1. Натисніть вкладку «Pull requests» вашого репозиторію «hello-world».
2. Натисніть «New pull request».

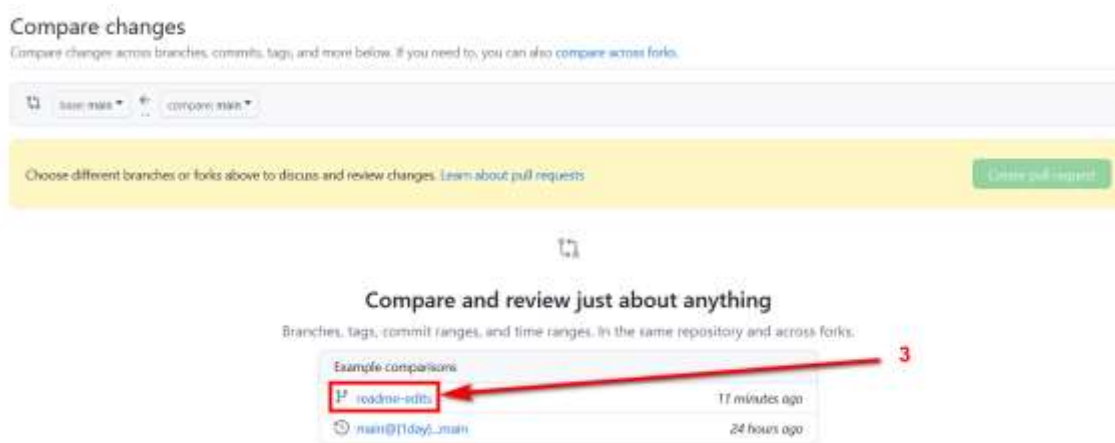


Рисунок 1.12 – Вибір гілки

3. У блоку «Example Comparisons» виберіть гілку, яку ви створили, «readme-edits», для порівняння з «main» (основною).

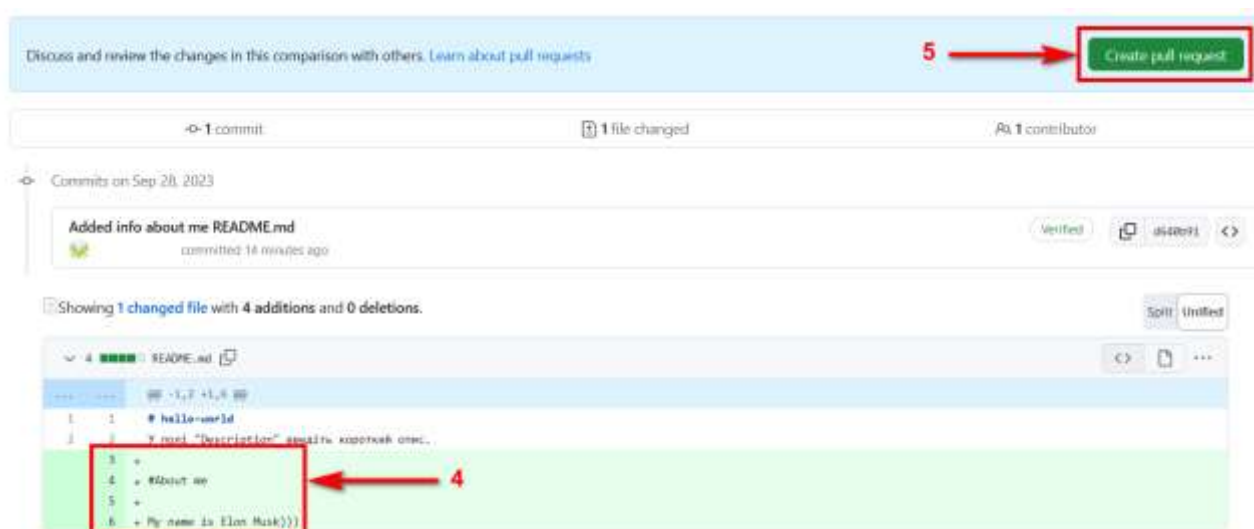


Рисунок 1.13 – Перегляд внесених змін

4. Перегляньте ваші зміни у різницях на сторінці порівняння і переконайтеся, що вони відповідають тому, що ви хочете подати.

5. Натисніть «Create pull request».

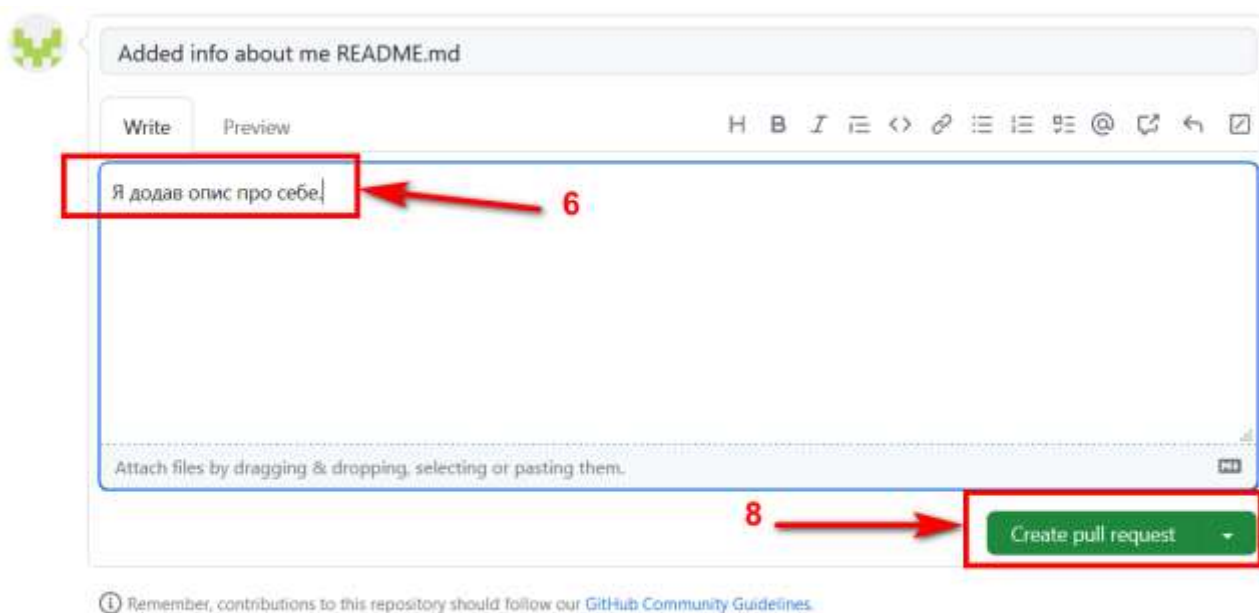


Рисунок 1.14 – Додавання опису та «Create pull request»

6. Дайте своєму запиту на включення змін заголовок і напишіть короткий опис ваших змін. Ви можете включити емодзі та перетягувати зображення та гіфки.

7. Необов'язково, справа від вашого заголовка та опису, клацніть біля Рецензентів, Відділених, Міток, Проектів чи Плана, щоб додати одну з цих опцій до вашого запиту на включення змін. Зараз не обов'язково додавати жодну з них, але ці опції пропонують різні способи співпраці через запити на включення змін. Докладніше про це дивіться в розділі «About pull requests».

8. Натисніть «Create pull request».

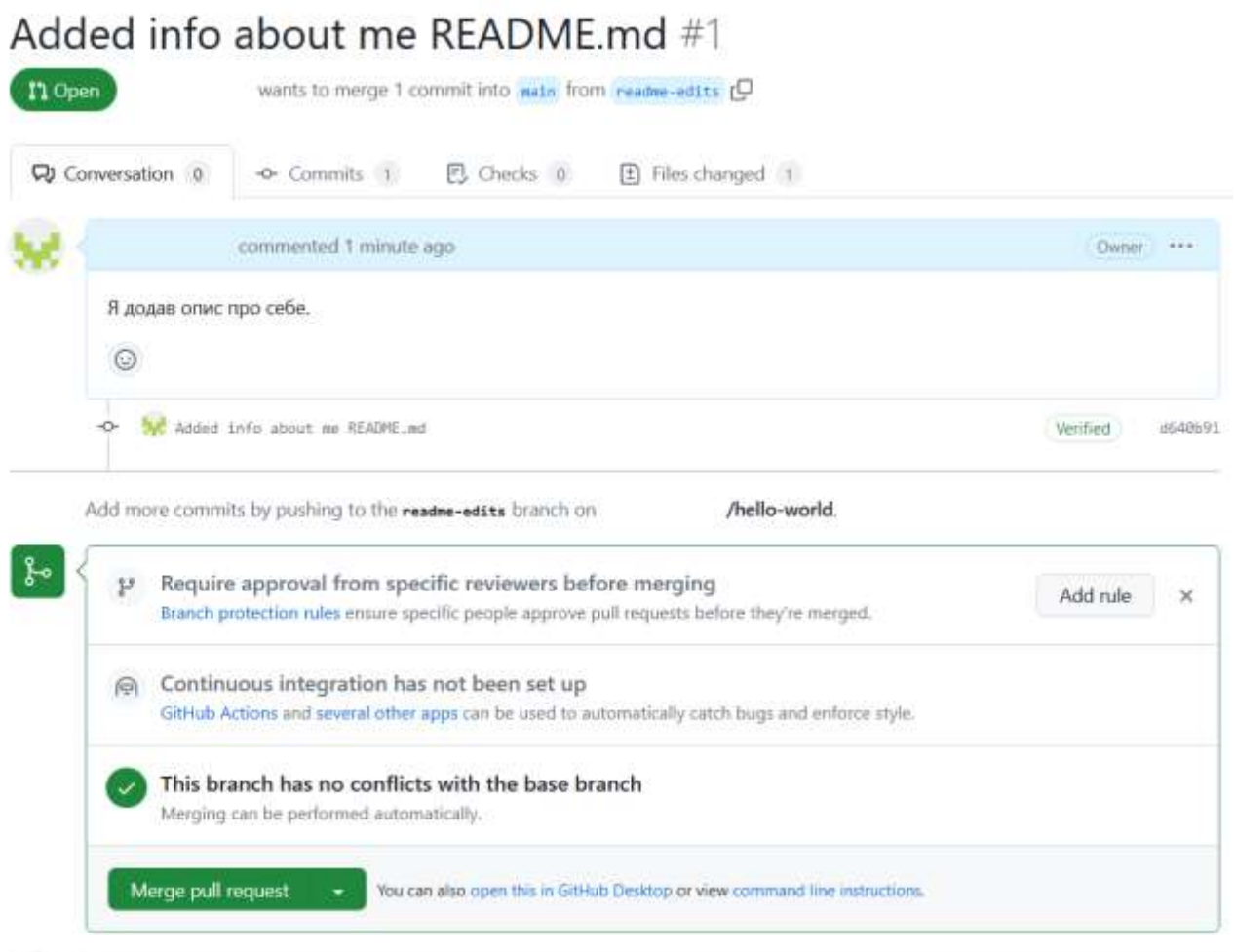


Рисунок 1.15 – Перевірка нових даних з актуальними в гілці «main»

Тепер ваші співпрацівники можуть переглянути ваші правки і надати пропозиції.

**Крок 6.** Об'єднання вашого запиту на включення змін «Merging your pull request».

На цьому останньому кроці ви об'єднаєте вашу гілку «readme-edits» з головною гілкою. Після об'єднання вашого запиту на включення змін, зміни у вашій гілці «readme-edits» будуть включені в головну гілку.

Іноді запит на включення змін може внести зміни в код, які конфліктують з існуючим кодом у головній гілці. Якщо є конфлікти, GitHub повідомить вас про конфліктний код і не дозволить об'єднання, поки конфлікти не будуть вирішені. Ви можете зробити коміт, який вирішує конфлікти, або використовувати коментарі в запиті на включення змін для обговорення конфліктів з членами вашої команди.

У цьому кроці не повинно бути конфліктів, тому Ви готові об'єднати вашу гілку з головною гілкою.

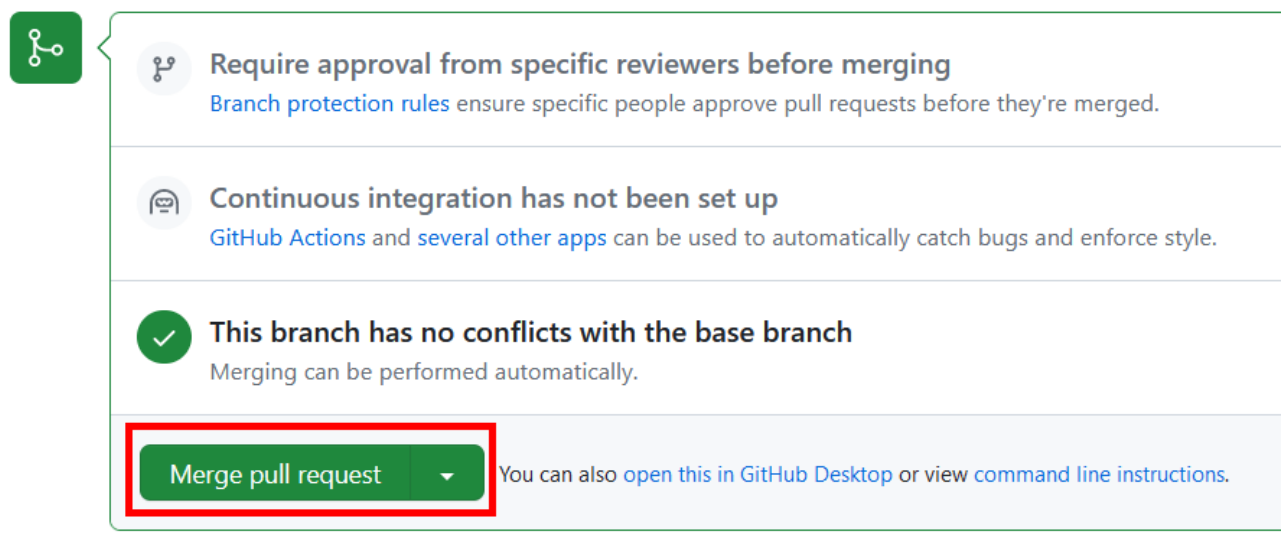


Рисунок 1.16 – Об'єднання змін з основною гілкою»Merge pull request»

1. Внизу запиту на включення змін натисніть «Merge pull request», щоб об'єднати зміни у головну гілку.



Рисунок 1.17 – Підтвердження змін «Confirm merge»

2. Після натискання на кнопку «Confirm merge» Ви отримаєте повідомлення про те, що запит був успішно об’єднаний і запит був закритий.



Рисунок 1.18 – Видалення гілки «Delete branch»

3. Натисніть «Delete branch». Тепер, коли ваш запит на включення змін об’єднаний, і ваші зміни знаходяться в головній гілці, ви можете безпечно видалити гілку «readme-edits».

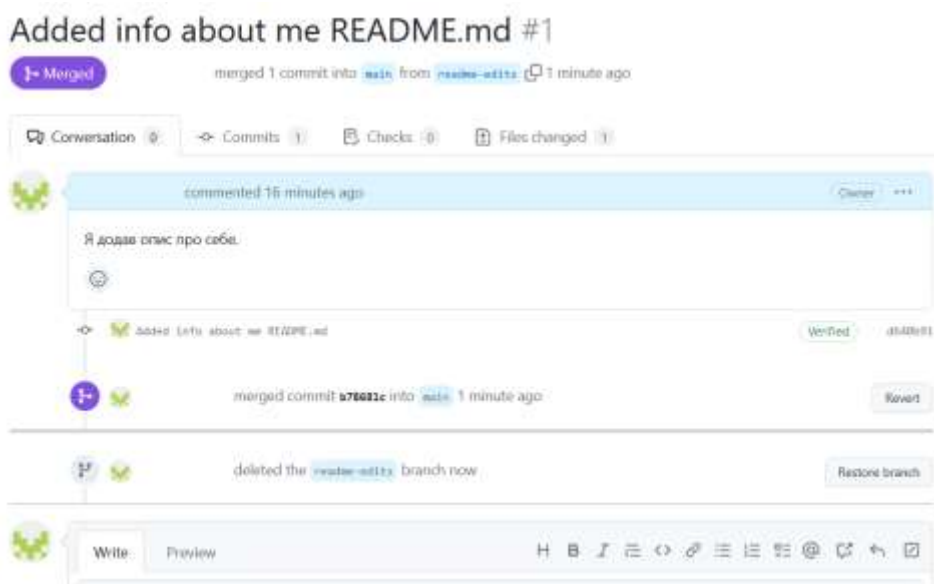


Рисунок 1.19 – Інформація про успішне об’єднання та видалення гілки

Якщо ви хочете внести більше змін у свій проект, ви завжди можете створити нову гілку і повторити цей процес.

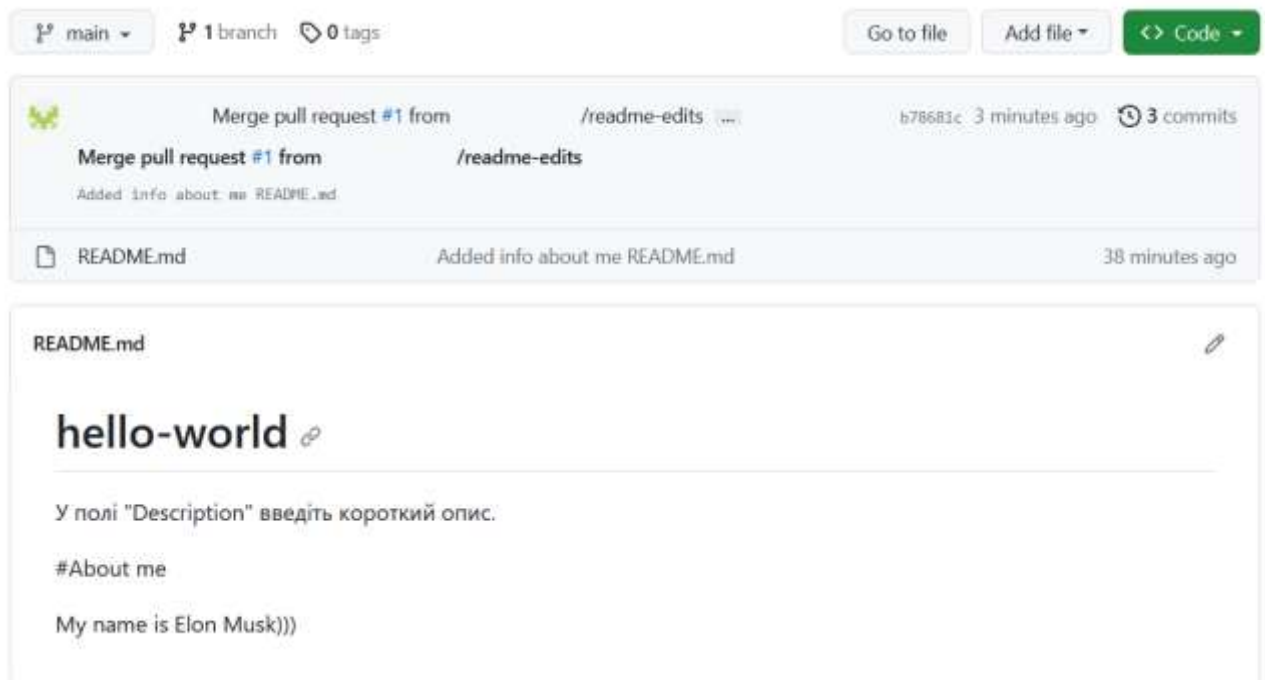


Рисунок 1.20 – Додаткова інформація про об’єднання на вкладці Code

**Крок 7.** Тепер необхідно перейти у термінал. Це можна зробити двома шляхами:

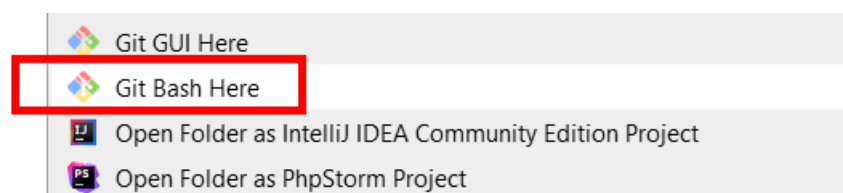


Рисунок 1.21 – Запуск Git Bash в папці з необхідним для завантаження файлом

– Відкрити Git Bash через exe файл та через команду `cd` перейти в необхідну папку, в якій знаходиться код, який необхідно завантажити в репозиторій.

– Або відкрийте необхідну папку, в якій міститься код для завантаження в репозиторій. Далі на вільному місці натисніть праву кнопку миші та оберіть варіант «Git Bash Here».

Після того як ми опинилися в необхідній папці треба обрати, якщо хочете задати одне (в нашому випадку одне) ім'я користувача для всіх репозиторіїв на комп'ютері, напишіть в командному рядку команду:

```
git config – global user.name «<ваше_ім'я>»
```

Замініть <ваше\_ім'я> на своє ім'я в лапках. Можете написати все, що завгодно. Якщо хочете задати ім'я лише одному репозитору, то видаліть з команди слово global.

Тепер напишіть свою адресу електронної пошти. Простежте, щоб вона збігалася з адресою, вказаною під час реєстрації на GitHub.

```
git config – global user.email «електронна_адреса@email.com»
```

Далі необхідно ініціалізувати папку, для цього виконайте наступну команду:

```
git init
```

Після ініціалізації необхідно завантажити наш файл README.md з репозитору GitHub в папку, для цього використайте команду «git clone» та посилання на ваш репозиторій.

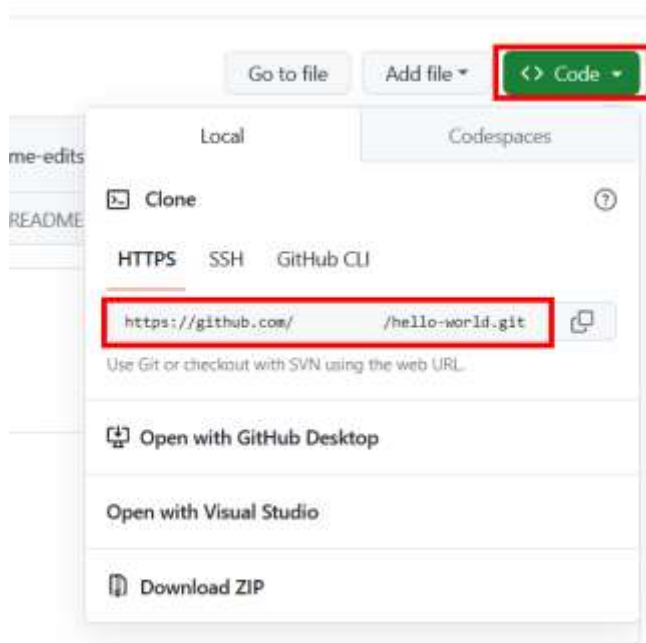


Рисунок 1.22 – Місцезнаходження посилання на репозиторій

Щоб взяти посилання на репозиторій, можна натиснути на зелену кнопку code та скопіювати посилання в один клік.

```
git clone https://github.com/Ваш_нікнейм/hello-world.git
```

Коли файл буде завантажений до вашої папки, його необхідно відкрити та видалити напис «У полі «Description» введіть короткий опис». Замість нього впишіть назву своєї групи та нижче після «#About me» додайте ще якоїсь інформації, наприклад, останні новини минулого дня та збережіть зміни.

### **Завдання для самостійного виконання**

Необхідно нижче ознайомитися з додатковою інформацією про команди. Також треба виконати наступні завдання.

1. Створити нову гілку в репозиторії.
2. Перейти до цієї гілки.
3. Завантажити на GitHub файл README.md зі змінами та перевірити його в репозиторії.
4. Створити файл або файли в папці, наповнити його, можливо, якимось кодом з лабораторної роботи по іншому предмету.
5. Завантажити файл або файли в цю саму нову гілку.
6. Виконати кроки, які робили вище з кроку №5 по №7.

## **Додаткова інформація:**

**Початкове налаштування.** Налаштування інформації про користувача для всіх локальних репозиторіїв

```
$ git config --global user.name "[ім'я]"
```

Встановлює ім'я, яке відображатиметься в полі автора у комітів, які ви виконуєте.

```
$ git config --global user.email "[адреса електронної пошти]"
```

Встановлює адресу електронної пошти, яка відображатиметься в інформації про коміти, які ви виконуєте.

**Внесення змін в репозиторій.** Перегляд змін та створення комітів (фіксація змін).

```
$ git status
```

Перераховує всі нові або змінені файли, які потребують фіксації.

```
$ git diff
```

Показує відмінності щодо внесених змін у ще не проіндексованих файлах.

```
$ git add [файл]
```

Індексує вказаний файл для наступного коміту.

```
$ git diff --staged
```

Показує різницю між проіндексованою та останньою зафіксованою версіями файлів.

```
$ git reset [файл]
```

Скасовує індексацію вказаного файлу, зберігаючи його вміст.

```
$ git commit -m "[повідомлення з описом]"
```

Фіксує проіндексовані зміни та зберігає їх в історію версій.

**Створення репозиторію.** Створення нового репозиторію або отримання його за існуючою URL-адресою.

```
$ git init [назва проекту]
```

Створює новий локальний репозиторій із заданим ім'ям.

```
$ git clone [url-адреса]
```

Завантажує репозиторій разом із усією його історією змін.

**Колективна робота.** Іменовані серії коммітів та поєднання результатів роботи.

*\$ git branch*

Список іменованих гілок коммітів із зазначенням обраної гілки.

*\$ git branch [ім'я гілки]*

Створює нову гілку.

*\$ git switch -c [ім'я гілки]*

Перемикається на вибрану гілку та оновлює робочу директорію до її стану.

*\$ git merge [ім'я гілки]*

Вносить зміни вказаної гілки в поточну гілку.

*\$ git branch -d [ім'я гілки]*

Видаляє вибрану гілку.

**Операції з файлами.** Переміщення та видалення версій файлів репозиторію.

*\$ git rm [файл]*

Видаляє конкретний файл із робочої директорії та індексує його видалення.

*\$ git rm --cached [файл]*

Забирає конкретний файл із контролю версій, але фізично залишає його на своєму місці.

*\$ git mv [оригінальний файл] [нове ім'я]*

Переміщує та перейменовує вказаний файл, одразу індексує його для наступного комміту.

**Перегляд історії.** Перегляд та вивчення історії змін файлів проекту.

*\$ git log*

Історія коммітів для поточної гілки.

*\$ git log --follow [файл]*

Історія змін конкретного файлу, включаючи його перейменування.

*\$ git diff [перша гілка]...[друга гілка]*

Показує різницю між змістом комітів двох гілок.

*\$ git show [коміт]*

Виводить інформацію та показує зміни у вибраному коментарі.

**Ігнорування деяких файлів.** Виключення тимчасових та вторинних файлів та директорій.

*\*.log*

*build/*

*temp-\**

Git ігноруватиме файли та директорії, перелічені у файлі `.gitignore` за допомогою wildcard синтаксису.

*\$ git ls-files --others --ignored --exclude-standard*

Список всіх ігнорованих файлів у поточному проекті.

**Відкат комітів.** Видалення помилок та коригування створеної історії.

*\$ git reset [коміт]*

Скасує всі коміти після заданого, залишаючи всі зміни у робочій директорії.

*\$ git reset --hard [коміт]*

Скидає всю історію разом із станом робочої директорії до вказаного коміту.

**Збереження фрагментів.** Збереження та відновлення незавершених змін.

*\$ git stash*

Тимчасово зберігає всі незафіксовані зміни файлів, що відстежуються.

*\$ git stash pop*

Відновлює стан раніше збережених версій файлів.

*\$ git stash list*

Виводить список усіх тимчасових збережень.

*\$ git stash drop*

Скидає останні тимчасово збережені зміни.

**Синхронізація з віддаленим репозиторієм.** Реєстрація видаленого репозиторію та обмін змінами.

*\$ git fetch [віддалений репозиторій]*

Завантажує всю історію з віддаленого репозиторію.

*\$ git merge [віддалений репозиторій]/[гілка]*

Вносить зміни з гілки віддаленого репозиторію у поточну гілку локального репозиторію.

*\$ git push [віддалений репозиторій] [гілка]*

Завантажує всі зміни локальної гілки у віддалений репозиторій.

*\$ git pull*

Завантажує історію з віддаленого репозиторію та поєднує її з локальною.

pull = fetch + merge

### **Зміст звіту**

1. Номер, тема та мета лабораторної роботи.
2. Відповіді на контрольні запитання.
3. Порядок і результати виконання самостійного завдання (скріншоти та текстовий опис кроків виконання завдань).
4. Висновки до роботи.

### **Контрольні питання**

1. Що таке Git Bash і як він пов'язаний із GitHub?
2. Які основні переваги використання GitHub для спільної роботи над проектами?
3. Які кроки потрібно виконати для створення нового репозиторію на GitHub та додавання змін до нього?
4. Які основні компоненти GitHub, що використовуються при роботі з репозиторіями та змінами в них?
5. Яким чином ви можете відкрити запит на включення змін у GitHub і як це сприяє співпраці у розробці програмного забезпечення?